

TypeScript Type Annotations with Examples

This document explains TypeScript type annotations with practical examples. Type annotations help developers define the expected data types in variables, functions, arrays, objects, and interfaces.

1. Basic Variable Type Annotations

```
let name: string = "Bala";
let age: number = 30;
let isTester: boolean = true;
```

2. Array Type Annotations

```
let skills: string[] = ["Playwright", "Selenium"];
let marks: Array<number> = [90, 95, 100];
```

3. Function Parameter Type Annotations

```
function greet(name: string) {
  console.log(name);
}
```

4. Function Return Type Annotations

```
function add(a: number, b: number): number {
  return a + b;
}
```

5. Object Type Annotation

```
let employee: {
  name: string;
  age: number;
} = {
  name: "Bala",
  age: 30
};
```

6. Interface Type Annotation

```
interface Employee {
  name: string;
  age: number;
}

const emp: Employee = {
  name: "Bala",
  age: 30
};
```

7. Union Types

```
let value: string | number;

value = "Playwright";
value = 100;
```

8. Optional Properties

```
interface User {
  name: string;
  age?: number;
}
```

9. Function Type Annotation

```
const multiply: (a: number, b: number) => number =
  (a, b) => {
    return a * b;
  };
```

10. Type Inference

```
let city = "Chennai";

// TypeScript automatically infers the type as string
```

11. Any Type

```
let data: any = "Hello";

data = 100;
data = true;
```

12. PDF Parser Example

```
interface TextRun {
  T: string;
}

interface TextBlock {
  R: TextRun[];
}

interface Page {
  Texts: TextBlock[];
}

interface PdfContents {
  Pages: Page[];
}

const pdfContents: PdfContents = getPdfData();
```

Type annotations in TypeScript improve readability, maintainability, IntelliSense support, and compile-time error checking. They are an essential part of writing scalable and reliable TypeScript applications.